

MyHealth Care — Project Checklist

Senior Project · University of Bahrain, College of IT

Ali Mohamed Jaafar Mohamed (202208244) · Mohammed A. Redha Meftah (202209027) · Supervisor: Dr. Amal Ghanim

Repository: github.com/Meftah7/myhealthcare_v2

Status: 116 / 118 tasks done · 74 automated tests passing · Windows + Web builds working

Phase 0 — Project Setup

Flutter

- ☒ Create Flutter project (Windows, Android, Web, iOS)
- ☒ App name "MyHealth Care" + app icons (all platforms)
- ☒ Add all packages (Riverpod, Drift, go_router, dio, fl_chart, ...)
- ☒ Create folder structure (feature-first + domain / data / services)
- ☒ Configure theme (colour scheme, clinical status colours, fonts, spacing)
- ☒ Configure routing (go_router, one shell per role)
- ☒ Dependency registry (ProviderScope)

Local database (no Firebase — offline-first)

- ☒ Drift + SQLite working on Windows
- ☒ Drift + SQLite working on Web (WASM + OPFS)
- ☐ Drift + SQLite on Android — blocked (emulator needs hypervisor driver, one admin step)
- ☒ `Result` / `Failure` types (no exceptions across layers)

Project hygiene

- ☒ Strict lint rules, `flutter analyze` clean
- ☒ Git repository + `.gitignore` (no API keys ever committed)
- ☒ README (run, seed, add AI key)

☒ **Done when:** app builds and runs on Windows and Web; local database opens and stores data.

Phase 1 — Database

Tables (16 total)

- ✓ Users, Patient Profiles, Staff Profiles, Departments
- ✓ Appointments, Schedule Templates, Reminders
- ✓ Medical Records, Lab Values, Vitals, Medications
- ✓ AI Summaries, Staff Tasks, Risk Flags
- ✓ Audit Log, App Settings

Domain layer

- ✓ Entities (User, Patient, Staff, Appointment, Record, Vitals, Medication, ...)
- ✓ Repository interfaces (no database imports — swap point for a future server)
- ✓ Repository implementations
- ✓ Register everything in the dependency registry

Relations (enforced by foreign keys)

- ✓ Patient → Appointments, Records, Vitals, Medications
- ✓ Appointment → Patient, Staff, Department
- ✓ Staff → Appointments, Schedule Templates, Tasks
- ✓ Record → Lab Values · Risk Flag → Patient · Task → Staff + Patient
- ✓ Department → Staff

Synthetic data (no real patients)

- ✓ Clinical vocabulary (names, conditions, medications, lab ranges)
- ✓ Seeder: 5 departments, 12 doctors, 60 patients, ~2 years of history
- ✓ Idempotent re-seed + reset demo data
- ✓ Export database diagram (ERD) to `docs/erd.png`

✓ **Done when:** all tables exist, all relations are enforced, and the database is fully populated with realistic data.

Phase 2 — Authentication & Patient Core

Authentication

- ☒ Salted password hashing (PBKDF2-HMAC-SHA256)
- ☒ Login screen
- ☒ Patient self-registration
- ☒ Session persistence (stay logged in after restart)
- ☒ Role-gated routing (patient / staff / admin can't cross areas)
- ☐ Logout + switch-user UI polish (backend done)

Uses local password auth, not phone/OTP. Real SMS verification would be added for production.

Patient screens

- ☒ Home (next appointment, active medications, AI summary card)
- ☒ Health timeline (records + vitals merged, grouped by month)
- ☒ Timeline filters + search
- ☒ Record detail (lab values flagged vs reference range)
- ☒ Vitals list + manual entry
- ☒ Vitals charts (blood pressure, weight, glucose, heart rate)
- ☒ Medications list (current / past)
- ☒ Profile & settings
- ☒ Shared widgets (empty / error / loading / confirm)

Deferred

- ☐ PDF report import — pick file, extract text, save as a record
- ☐ Attachment viewer

☒ **Done when:** a patient can log in and browse their entire history.

Phase 3 — AI Health Summary (Research Question 1)

- ✓ AI service interface + response models
 - ✓ Patient context builder (with a token budget)
 - ✓ Mock AI service — deterministic, derived from the patient's real data
 - ✓ Summary prompt + strict JSON contract + tolerant parser
 - ✓ Live AI service — Google Gemini (free tier), retry, timeout
 - ✓ Fallback: live AI fails → drop to mock automatically
 - ✓ API key in secure storage (never committed)
 - ✓ Provider picks mock vs live from settings
 - ✓ Summary cache keyed on the input hash
 - ✓ AI summary screen (narrative + key events + trends + red flags)
 - ✓ Summary card on the home screen + regenerate
 - ✓ Key events shown as markers on the timeline
 - ✓ Trends link to the matching vitals chart
 - ✓ "Not medical advice" safety banner on every AI screen
 - ✓ Store model id + prompt version + timestamp with each summary
- ✓

Done when: the app produces a useful health summary with AI on **or** off.

Phase 4 — No-Show Model & Smart Scheduling (Research Question 2)

Machine learning (Python, trained offline)

- ✓ Python environment + requirements
- ✓ Generate synthetic training dataset (~8,500 appointments)
- ✓ Feature engineering — 11 features
- ✓ Train class-balanced logistic regression
- ✓ Evaluate → `docs/ml_results.md` + ROC curve
- ✓ Export model weights to JSON (bundled as an asset)
- ✓ Baseline comparison (majority class, simple heuristic)

Result: ROC-AUC 0.75, recall 0.62 — beats both baselines.

On-device inference (Dart)

- ✓ Feature extractor (identical encoding to Python)
- ✓ Predictor — scale, sigmoid, risk band
- ✓ Per-feature contribution breakdown (explainability)
- ✓ Python ↔ Dart parity test within 1×10^{-6}

Booking

- ✓ Open-slot generator (schedule templates minus booked)
- ✓ Booking wizard (department → doctor → date → slot → confirm)
- ✓ Slot ranking by predicted risk, each with a plain-language reason
- ✓ My appointments — cancel, reschedule
- ✓ Store predicted risk + band on the appointment
- ☐ Optional AI-written slot rationale — skipped (model reasons are enough)

Reminders

- ✓ Notification abstraction + in-app fallback (works on web too)
- ✓ Risk-adaptive reminder escalation (1 / 2 / 3 reminders by risk band)

✓ **Done when:** the booking flow recommends low-risk slots with real, reportable accuracy.

Phase 5 — Staff & Admin (Research Question 3)

Rule engine (works with AI switched off)

- ✓ Risk detection — abnormal vitals, critical labs, medication gaps, overdue follow-ups
- ✓ Severity scoring (info / warning / urgent) + de-duplication
- ✓ Rule-based task generator

Staff app

- ✓ Staff shell + navigation (Dashboard / Patients / Tasks / Schedule)
- ✓ Dashboard — today's schedule, risk flags, task preview, one-tap "scan panel"
- ✓ Patient search + list
- ✓ Patient chart (header, flags, medications, vitals, timeline)
- ✓ Add clinical note
- ✓ Prescribe medication
- ✓ Enter lab result (auto-classified against the reference range)
- ✓ AI task prioritisation, blended with the rule score
- ✓ Task board with a per-task rationale
- ✓ Week schedule grid
- ✓ Panel analytics — no-show rate, cancellation rate, utilisation

Admin dashboard

- ✓ User management — create staff, activate / deactivate, reset password
- ✓ Departments — create / rename / describe
- ☐ Schedule-template editor — deferred (templates are seeded for now)
- ✓ AI settings — API key, model, mock toggle, blend weight, re-seed
- ✓ System analytics + audit log viewer

✓ **Done when:** staff can run a full clinical workflow and admin can manage the platform.

Phase 6 — Testing & Results

- ✓ Unit tests — predictor, features, risk rules, password hashing
- ✓ Unit tests — repositories on an in-memory database
- ✓ Unit tests — AI parsing (valid, malformed, empty, timeout → safe error)
- ✓ Widget tests — login, timeline, booking, staff dashboard
- ✓ Integration test — seed → patient books → staff sees it
- ✓ Cross-platform smoke — Windows + Web release builds green
- ☐ Cross-platform smoke — Android (blocked on the hypervisor driver)
- ☐ Usability study — 5–8 people, SUS questionnaire (protocol ready, needs participants)
- ✓ Performance — timeline load, AI latency, cold start, database size
- ✓ Accessibility — contrast, tap targets, labels, text scaling

Result: 74 tests, all passing. flutter analyze clean.

✓ **Done when:** every number in the report has a test or measurement behind it.

Phase 7 — Report & Presentation (in progress)

- ☐ Architecture, use-case, sequence diagrams
- ☐ Screenshots — all screens, both roles, light + dark
- ☐ Results chapter — ML metrics, baselines, SUS, performance
- ☐ Demo script, rehearsed offline with the mock AI
- ☐ Presentation deck + rehearsal

☒ **Done when:** the report and defense materials are complete.

Open Items

Item	Status	To close
Android build	Blocked	Run the AEHD hypervisor driver installer (admin), or use a physical phone
PDF report import	Deferred	Wire file picker → text extractor → new record
Schedule-template editor	Deferred	Add a schedule-template repository
Usability study results	Pending	Run the protocol in docs/usability_study.md with 5–8 people
Live Gemini output	Dormant	Paste a free API key in Admin → AI Settings, turn Mock mode off